PROJECT DOCUMENTATION

# RIG Query Agent

*An AI-Powered Conversational Assistant for Offshore Rig Operations*

---

**PREPARED BY** Alan S Philip

Pranav Santosh Nair

**ROLE** Intern

**ORGANIZATION** Tech Mahindra

**DATE** August 2026

**Frontend:** https://rig-agent-demo.vercel.app/

**Backend API:** https://rigagent-demo.onrender.com

**Repository:** https://github.com/alansphilip/RigAgent-Demo

## Table of Contents

# 1. Executive Summary

The RIG Query Agent is an AI-powered conversational assistant built for offshore oil rig operations. It allows rig operators, engineers, and supervisors to query live operational data — including equipment manuals, work packs, shift rosters, maintenance procedures, and inspection checklists — using plain natural language instead of navigating complex ERP dashboards or paper-based systems.

The agent is powered by Google Gemini (LLM), a TF-IDF knowledge retrieval engine (RAG), and a structured SQLite operational database. It is deployed as a full-stack web application, with a React frontend hosted on Vercel and a FastAPI backend hosted on Render.

> *This is a demonstration / proof-of-concept system, built and deployed independently as part of an internship project. All operational data used is realistic synthetic data seeded into the database for demonstration purposes.*

# 2. Project Background & Motivation

## 2.1 Problem Statement

Offshore oil rigs are complex, high-pressure operational environments. Rig personnel routinely need to:

- Look up equipment manuals and specifications mid-operation
- Check which work packs are active or pending
- Find out who is currently on shift and their status
- Retrieve standard operating procedures and inspection checklists
- Answer ad-hoc operational questions quickly and accurately

Existing solutions require navigating bulky ERP systems, searching through paper binders, or waiting on supervisors — all slow and error-prone in time-critical situations.

## 2.2 Proposed Solution

The RIG Query Agent provides a single conversational interface where any crew member can type a natural-language question and receive an accurate, AI-generated answer drawing from:

- The rig's live operational database (work packs, shifts, procedures)
- An equipment knowledge base (manuals, specs, maintenance guides)
- General engineering and operational knowledge via Gemini AI

## 2.3 Project Scope

This is a demonstration / proof-of-concept system built as part of my internship at Tech Mahindra. The data used is realistic synthetic data seeded into a relational database for demonstration purposes.

# 3. System Overview

The RIG Query Agent operates as a three-layer intelligent system: a browser-based chat interface, a FastAPI backend that classifies and answers each query, and a response stream sent back to the user in real time.

```
USER (Browser / Chat UI)
     |  Natural language query
FRONTEND (React + Vite — Vercel)
     |  HTTP / SSE streaming
BACKEND API (FastAPI — Render)
     |-- Intent Router  -->  Identifies query type
     |-- Agent Tools    -->  Queries SQLite database
     |-- RAG Engine     -->  Retrieves equipment knowledge
     `-- LLM Client     -->  Generates AI response (Gemini)
     |
RESPONSE streamed back word-by-word to user
```

### How a Query Is Processed

- User types a message (e.g. “Who is on the current shift?”)
- Frontend sends the query to the backend via Server-Sent Events (SSE) streaming
- The Intent Router classifies the query into one of: shift, work_pack, procedure, equipment, checklist_search, checklist_pdf, greeting, or general
- The appropriate Agent Tool retrieves structured data from SQLite (e.g. shift records)
- The retrieved data is passed to Gemini AI as context
- Gemini generates a natural-language response using the data and its own knowledge
- The response is streamed word-by-word to the frontend for a near-instant feel

## 4. Key Features & Capabilities

### 4.1 Natural Language Understanding

Users can ask questions in plain English — no commands or special syntax required.

| Example Query | What Happens |
|---|---|
| Who is on duty right now? | Returns current active shift operators |
| Show all active work packs | Lists all in-progress / active maintenance work packs |
| What is a blowout preventer? | Returns equipment manual + AI explanation |
| Generate Mud Pump checklist PDF | Creates a downloadable PDF inspection checklist |
| List completed procedures | Returns completed SOPs from the database |
| How does a top drive work? | AI explains from the equipment knowledge base |
| Hello | Friendly greeting and capability summary |

### 4.2 Streaming Responses

Responses appear word-by-word in real time (similar to ChatGPT) using Server-Sent Events, removing the perception of waiting even for longer answers.

### 4.3 PDF Checklist Generation

The agent can generate downloadable inspection checklists as formatted PDF files directly from the database, ready for field use.

### 4.4 Equipment Knowledge Base

A knowledge base of 20 offshore drilling equipment entries, including Mud Pump, BOP, Top Drive, Rotary Table, Drill Pipe, Choke Manifold, Mud Motor, Draw Works, Iron Roughneck, Shale Shaker, Degasser, and Wireline.

### 4.5 Live Operational Database

Realistic operational data seeded into the system, including:

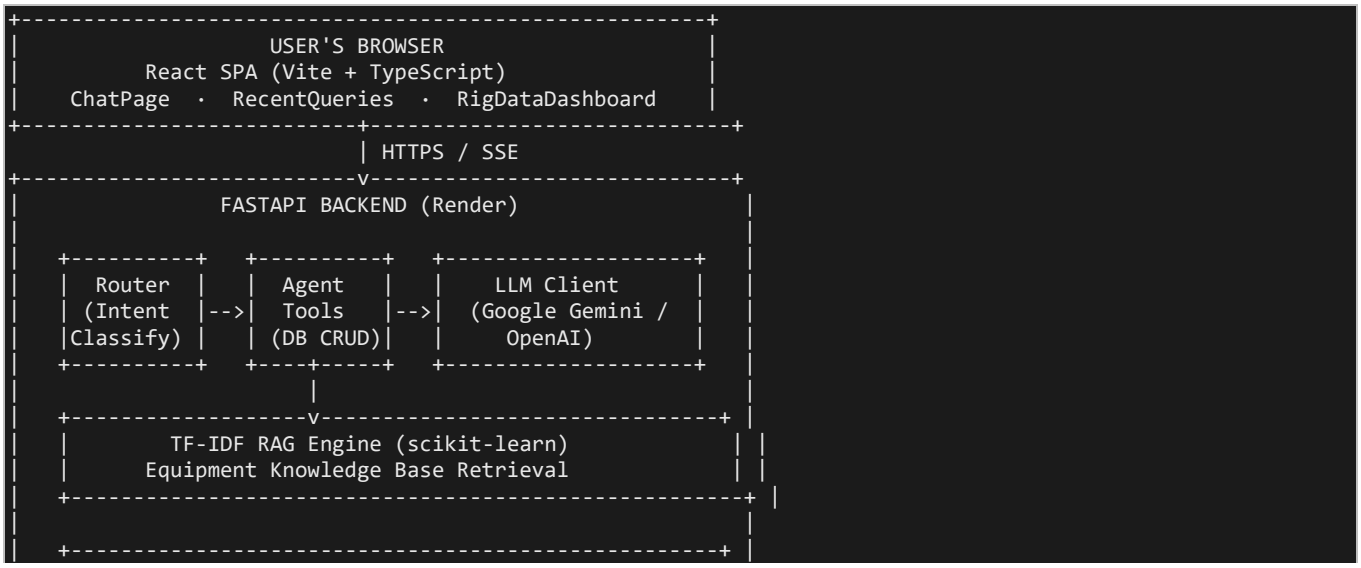
- 15 Work Packs (Active, In Progress, Completed, Pending)
- 40 Standard Operating Procedures
- 244 Operations
- 20 Shift records (Morning, Afternoon, Night)
- 20 Checklists comprising 176 inspection items

### 4.6 Fallback Intelligence

Even without rig-specific context, the agent answers general questions using Gemini's broad training knowledge, making it a fully capable general assistant deployed in a rig environment.

## 5. System Architecture

### 5.1 High-Level Architecture



```
|   |              SQLite Database                      | |
|   |  WorkPacks · Procedures · Shifts                 | |
|   |  Checklists · Operations · EquipmentKB            | |
|   +-------------------------------------------------+ |
+---------------------------------------------------------+
```

### 5.2 Backend Module Structure

```
backend/
|-- main.py          # FastAPI app, endpoints, LLMClient, query handler
|-- router.py        # Intent classification (regex-based NLP routing)
|-- agent_tools.py   # DB query functions for each intent type
|-- rag.py           # TF-IDF knowledge retrieval engine
|-- database.py      # SQLAlchemy ORM models & session management
|-- models.py        # Pydantic request/response schemas
|-- setup_db.py      # Database seeder (synthetic data)
`-- requirements.txt # Python dependencies
```

### 5.3 Frontend Module Structure

```
frontend/src/
|-- pages/
|   |-- ChatPage.tsx          # Main chat interface with SSE streaming
|   |-- RecentQueriesPage.tsx # Query history sidebar
|   `-- RigDataPage.tsx       # Live pump telemetry dashboard
|-- components/               # Navigation and layout components
`-- types/index.ts             # TypeScript type definitions
```

## 6. Technology Stack

### 6.1 Backend

| Component | Technology | Purpose |
|---|---|---|
| Web Framework | FastAPI (Python 3.11) | REST API + SSE streaming endpoints |
| ORM | SQLAlchemy 2.0 | Database abstraction layer |
| Database | SQLite | Lightweight relational DB for operational data |
| LLM Provider | Google Gemini 1.5 Flash | Natural language generation |
| RAG Engine | scikit-learn TF-IDF | Equipment knowledge retrieval |
| PDF Generation | ReportLab | Inspection checklist PDF export |
| ASGI Server | Uvicorn | Production-grade async server |

### 6.2 Frontend

| Component | Technology | Purpose |
|---|---|---|
| Framework | React 18 + TypeScript | Component-based UI |
| Build Tool | Vite | Fast bundler and dev server |
| Routing | React Router v6 | Client-side navigation |

| Component | Technology | Purpose |
|---|---|---|
| Styling | Tailwind CSS | Utility-first CSS framework |
| Markdown | react-markdown + remark-gfm | Renders AI responses with formatting |
| Icons | Lucide React | Icon set |

### 6.3 Infrastructure

| Service | Provider | Plan |
|---|---|---|
| Backend Hosting | Render | Free Web Service (512MB RAM) |
| Frontend Hosting | Vercel | Hobby (Free) |
| Source Control | GitHub | Public Repository |
| LLM API | Google AI Studio | Free tier (15 req/min, 1M tokens/day) |

## 7. Data Model & Knowledge Base

### 7.1 Database Schema (SQLite)

```
WorkPack              Procedure             Operation
--------              ---------             ---------
id                    id                    id
code (WP001...)       code (P001...)        name
name                  name                  step_order
status                status                status
priority              assigned_to           procedure_id --> Procedure
created_date          work_pack_id --> WorkPack
description

Shift                 Checklist             ChecklistItem
-----                 ---------             -------------
id                    id                    id
operator_name         name                  description
shift_type            equipment             is_required
login_time            created_date          step_number
logout_time           work_pack_id          checklist_id --> Checklist
status
date

EquipmentKB
-----------
id
equipment_name
content               (Full-text equipment manual)
```

### 7.2 Seeded Data Summary

| Table | Records | Notes |
|---|---|---|
| Work Packs | 15 | WP001–WP015, mixed statuses |
| Procedures | 40 | P001–P040, linked to work packs |

| Table | Records | Notes |
|---|---|---|
| Operations | 244 | Step-by-step task records |
| Shifts | 20 | Morning / Afternoon / Night |
| Checklists | 20 | Equipment-specific |
| Checklist Items | 176 | Inspection steps |
| Equipment KB | 20 | Full manuals per equipment |

### 7.3 Equipment Knowledge Base

The knowledge base contains detailed technical entries for 20 pieces of equipment, including: Mud Pump, Blowout Preventer (BOP), Top Drive, Rotary Table, Drill Pipe, Choke Manifold, Mud Motor, Draw Works, Kelly System, Hook & Swivel, Shale Shaker, Degasser, Iron Roughneck, Wellhead, Riser System, Accumulator Unit, Standpipe Manifold, Cementing Unit, Wireline Unit, and Heave Compensator.

## 8. AI & Intelligence Layer

### 8.1 Intent Routing

The router module classifies every incoming query, using regex pattern matching, into one of eight intents:

| Intent | Trigger Examples | Action |
|---|---|---|
| greeting | “hello”, “hi”, “who are you” | Warm greeting + capability summary |
| shift | “who is working”, “on duty” | Query Shift table → Gemini formats |
| work_pack | “active work packs”, “WP001 status” | Query WorkPack table → Gemini formats |
| procedure | “completed procedures”, “P003 status” | Query Procedure table → Gemini formats |
| equipment | “what is a BOP”, “mud pump specs” | RAG retrieval → Gemini explains |
| checklist_search | “show checklists”, “checklist for BOP” | Query Checklist table → Gemini formats |
| checklist_pdf | “download checklist”, “generate PDF” | ReportLab PDF → download link |
| general | Everything else | RAG (if relevant) + Gemini answers freely |

### 8.2 RAG (Retrieval-Augmented Generation)

The RAG engine uses TF-IDF vectorization (scikit-learn) to retrieve relevant equipment knowledge:

- At startup, all equipment KB entries are chunked (~200 words/chunk) and vectorized
- On query, the query is vectorized and cosine similarity is computed against all chunks
- Top chunks with a similarity score above 0.1 are returned as context; low-score chunks are discarded to avoid irrelevant content

- Context is passed to Gemini, which generates a natural answer

> *Why TF-IDF instead of vector embeddings? Render's free tier has a 512MB RAM limit. Embedding models (sentence-transformers, FAISS) require 400MB+ just to load. TF-IDF achieves good retrieval accuracy at approximately 5MB memory footprint.*

### 8.3 LLM Integration (Google Gemini)

The LLM Client supports two providers with automatic fallback:

```
Priority 1: Google Gemini (GEMINI_API_KEY set)
    Model: gemini-1.5-flash
    Free tier: 15 req/min, 1M tokens/day

Priority 2: OpenAI (OPENAI_API_KEY set)
    Model: gpt-4o-mini (configurable)

Priority 3: Smart structured fallback (no key)
    Returns formatted DB data without LLM
```

Two system prompts are used: a rig-expert persona for all domain-specific queries, and an open-assistant persona for general, any-topic queries.

### 8.4 Streaming (Server-Sent Events)

The /query/stream endpoint uses Python async generators to stream responses: the backend computes the full answer, splits it into 3-word chunks, and sends each as an SSE event, which the frontend appends to the UI in real time — creating the word-by-word typing effect that makes responses feel immediate.

## 9. API Reference

**Base URL: https://rigagent-demo.onrender.com**

### POST /query

Standard query endpoint. Returns a full JSON response.

```
Request:  { "message": "Who is on duty?" }

Response:
{
  "answer": "Currently, 3 operators are on the Morning shift...",
  "pdf_url": null,
  "tool_used": "shift",
  "sources": null
}
```

### GET /query/stream?message={text}

Streaming SSE endpoint. Returns a token-by-token stream.

```
data: {"token": "Currently ", "done": false}
data: {"token": "3 operators ", "done": false}
data: {"token": "", "done": true, "tool_used": "shift"}
```

### Other Endpoints

| Endpoint | Purpose |
|---|---|
| GET /health | Health check |
| GET /ping | Keep-warm endpoint, called by UptimeRobot every 5 minutes |
| GET /checklist/{filename} | Download a generated PDF checklist file |
| GET /rig-data | Returns simulated pump telemetry data for the dashboard |
| GET /system-status | Returns system health and subsystem status |

## 10. Frontend User Interface

### 10.1 Pages

| Page | Route | Description |
|---|---|---|
| Chat | /chat | Main conversational interface |
| Recent Queries | /recent | History of past queries with timestamps |
| Rig Data | /rig-data | Live pump telemetry dashboard (simulated) |

### 10.2 Chat Interface Design

- Dark industrial theme with amber accent colour, consistent with offshore control-room aesthetics
- "Analysis Complete" header shown on each AI response
- Streaming responses — words appear progressively, word-by-word
- Markdown rendering — tables, bold text, bullet points, and code blocks render natively
- PDF download button appears inline when a checklist PDF is generated
- "Connecting to backend..." shown during cold start (Render free-tier wake-up)
- Automatic retry — pings backend every 2 seconds until alive (max 60 seconds)
- Fallback to standard REST call if SSE streaming fails

### 10.3 Local Storage

Recent queries are saved in browser local storage (max 50 items) and shown on the Recent Queries page for quick re-execution.

## 11. Deployment & Infrastructure

### 11.1 Deployment Architecture

```
GitHub (main branch)
    |
    |---> Render (auto-deploy on push)
    |       Backend: FastAPI + Uvicorn
```

```
    |       Build:  pip install -r requirements.txt && python setup_db.py
    |       Start:  uvicorn main:app --host 0.0.0.0 --port $PORT
    |
    `---> Vercel (auto-deploy on push)
            Frontend: React + Vite
            Build:    npm run build
            Output:   dist/
            Root:     frontend/
```

### 11.2 Key Environment Variables

| Variable | Platform | Description |
|---|---|---|
| GEMINI_API_KEY | Render | Google Gemini API key (required for AI) |
| OPENAI_API_KEY | Render | OpenAI API key (optional fallback) |
| DATABASE_URL | Render | SQLite path (default: sqlite:///./rig_query.db) |
| CORS_ORIGINS | Render | Allowed frontend origins |
| VITE_API_URL | Vercel | Render backend URL |

### 11.3 Cold Start Management

Render's free tier spins down after 15 minutes of inactivity, causing 30–50 second cold starts on the next request.

> *Mitigation: UptimeRobot (free) pings the backend's /ping endpoint every 5 minutes, keeping it warm at zero additional cost.*

## 12. Security Considerations

| Concern | Status | Notes |
|---|---|---|
| API Key exposure | Protected | Keys stored as Render environment secrets, never in code |
| CORS | Configured | Accepts requests only from the Vercel frontend origin |
| SQL Injection | Protected | All DB queries use SQLAlchemy ORM parameterized queries |
| HTTPS | Enforced | Render and Vercel both enforce HTTPS automatically |
| Authentication | Not implemented | Out of scope for this demo; production requires an auth layer |
| Rate Limiting | Not implemented | Gemini free tier enforces its own limits (15 req/min) |

> *This is a demonstration project. A production deployment would require user authentication, role-based access control (RBAC), audit logging, and integration with the rig's actual SCADA / ERP systems.*

# 13. Known Limitations & Future Roadmap

## 13.1 Current Limitations

| Limitation | Reason | Workaround |
|---|---|---|
| Database resets on redeploy | Render ephemeral filesystem | Use PostgreSQL for production |
| 512MB RAM limit | Render free tier | Prevents large embedding models |
| Cold start delay (30–50s) | Render spins down when idle | UptimeRobot ping |
| No real-time rig data | Demo uses synthetic data | Integrate with actual SCADA |
| No user authentication | Out of scope for demo | Add OAuth / JWT for production |

## 13.2 Future Enhancements

| Feature | Priority | Description |
|---|---|---|
| Real SCADA integration | High | Connect to actual rig sensor data feeds |
| User authentication | High | Role-based access (operator, supervisor, admin) |
| Vector embeddings | Medium | Qdrant / Pinecone for higher-quality RAG |
| Conversation memory | Medium | Remember previous messages within a session |
| Alert integration | Medium | Agent proactively notifies about critical alarms |
| Voice input | Low | Speech-to-text for hands-free rig use |
| Mobile app | Low | React Native for field tablets |
| Multi-language support | Low | Support for non-English-speaking crew |

# 14. Glossary

| Term | Definition |
|---|---|
| RAG | Retrieval-Augmented Generation — retrieving relevant documents and passing them as context to an LLM before generating a response |
| LLM | Large Language Model — an AI model trained on text to generate human-like responses (e.g. Gemini, GPT-4) |
| TF-IDF | Term Frequency–Inverse Document Frequency — a statistical method for finding relevant text based on keyword importance |
| SSE | Server-Sent Events — an HTTP protocol for one-way, real-time server-to-client streaming |
| BOP | Blowout Preventer — critical rig safety equipment that seals the wellbore in an emergency |
| Work Pack | A grouped collection of maintenance procedures for a specific rig activity |

| Term | Definition |
|---|---|
| SOP | Standard Operating Procedure — documented step-by-step instructions for a task |
| SCADA | Supervisory Control and Data Acquisition — an industrial system for monitoring and controlling equipment |
| Cold Start | The delay experienced when a cloud service restarts after being idle |

## Appendix A — Project Timeline

| Phase | Activities | Status |
|---|---|---|
| Phase 1 | Requirements gathering, architecture design | Complete |
| Phase 2 | Database design, data seeding, backend API | Complete |
| Phase 3 | Frontend UI development, chat interface | Complete |
| Phase 4 | Gemini LLM integration, intent routing | Complete |
| Phase 5 | Streaming responses, RAG engine | Complete |
| Phase 6 | Deployment (Render + Vercel), debugging | Complete |
| Phase 7 | General query support, system prompt tuning | Complete |